

A Closure-Based Lambda Interpreter

Reading `lambda1.py`

CS413

September 22, 2026

The evaluator's interface

```
def d0exp_evaluate\  
(dexp: d0exp, denv: d0env = ENVnil()) -> d0val:  
    """  
    [dexp] may contain free variables  
    """
```

- An expression describes a computation.
- An environment gives values to variable names.
- Evaluation produces a value, including a closure for a function.
- The interpreter uses call-by-value and lexical scope.

The type aliases in this file require Python 3.12 or later.

The expression language

Constructors	Meaning
D0Eint, D0Ebtf	Integer and Boolean literals
D0Evar	Variable lookup
D0Elam, D0Efix	Ordinary and recursive functions
D0Eapp	Function application
D0Elet	Local, nonrecursive binding
D0Eop1, D0Eop2	Integer operations and comparisons
D0Eif0	Boolean conditional
D0Epair, D0Epfst, D0Epsnd	Pairs and projections

The file consumes abstract syntax trees. It does not parse source text.

Constructing an expression

$$(\lambda x. x + 1) 41$$

```
increment = D0Elam("x",  
    D0Eop1("+1", D0Evar("x")))  
  
term = D0Eapp(increment, D0Eint(41))  
  
d0exp_evaluate(term)  # D0Vint(arg1=42)
```

D0Eapp constructs syntax. d0exp_evaluate executes it.

@dataclass supplies constructors, representations, and equality.

Expressions and values have different roles

```
@dataclass
class D0Elam(D0E000):
    arg1: dvar
    arg2: d0exp
    ctag = "D0Elam"
```

A lambda expression stores its parameter and body.

Other values are D0Vint, D0Vbtf, D0Vpair, and D0Vfix.

The evaluator dispatches with `isinstance`, rather than `ctag`.

```
@dataclass
class D0Vlam(D0V000):
    arg1: d0env
    arg2: D0Elam
    ctag = "D0Vlam"
```

A closure stores an environment together with that expression.

Environments are linked bindings

```
@dataclass(frozen=True)
class ENVcns(ENV000):
    arg1: dvar
    arg2: d0val
    arg3: d0env
    ctag = "ENVcns"

outer = ENVcns("x", DOVint(10), ENVnil())
inner = ENVcns("x", DOVint(20), outer)
```

ENVnil() ends the chain. Each new binding retains the outer chain.

The links are frozen. The referenced values and syntax trees are not frozen.

Lookup chooses the nearest binding

```
def d0env_search(denv: d0env, dvar: dvar) -> d0val:
  while True:
    if isinstance(denv, ENVcns):
      if dvar == denv.arg1:
        return denv.arg2
      else:
        denv = denv.arg3; continue
    else:
      return DOV000() # HX: this indicates an error
```

For the preceding environments, lookup of `x` gives 20 in inner, but still gives 10 in outer. A missing name yields `DOV000()`.

Evaluating a lambda creates a closure

Evaluator excerpt:

```
elif isinstance(dexp, D0Elam):  
    return D0Vlam(denv, dexp)  
elif isinstance(dexp, D0Efix):  
    return D0Vfix(denv, dexp)
```

- The closure captures the environment at the function's definition.
- Creating a closure does not evaluate its body.
- The body can use captured bindings when a later call executes it.

No substitution or copying of the function body occurs here.

Application of an ordinary closure

Excerpts from `f0_D0Eapp`:

```
dfun = d0exp_evaluate(dexp.arg1, denv)
darg = d0exp_evaluate(dexp.arg2, denv)
```

For a `D0Vlam` value:

```
dlam = dfun.arg2
denv_new = ENVcns(dlam.arg1, darg, dfun.arg1)
return d0exp_evaluate(dlam.arg2, denv_new)
```

The caller's environment evaluates the function and argument, in that order.

The **captured environment** evaluates the body, with the parameter added.

Lexical scope in a worked example

Readable notation for a nested AST:

```
let x = 10 in
  let f = (lambda y. x + y) in
    let x = 100 in
      f(2)
```

When f is defined Its closure captures $x = 10$.

When $f(2)$ is called The caller has $x = 100$.

When the body runs It finds $y = 2$, then captured $x = 10$.

Result: `D0Vint(12)`

A let evaluates its initializer before binding

Body of f0_D0Elet:

```
dval = d0exp_evaluate(dexp.arg2, denv)
denv_new = ENVcns(dexp.arg1, dval, denv)
return d0exp_evaluate(dexp.arg3, denv_new)
```

In D0Elet(name, initializer, body):

- The initializer uses the old environment.
- Only the body sees the new binding.
- Even an unused initializer executes.

Example: with outer `x = 10`, `let x = x+1 in x` yields 11.

The outer binding remains 10. This form of let is nonrecursive.

A recursive function names itself

```
@dataclass
class D0Efix(D0E000):
    arg1: dvar
    arg2: dvar
    arg3: d0exp
    ctag = "D0Efix"
```

`D0Efix("f", "x", body)` represents `fix  $f(x)$ . body`.

`arg1` names the function itself, `arg2` names its parameter,
and `arg3` holds its body.

Evaluation creates `D0Vfix(denv, dexp)`. A call adds the self binding.

Application of a recursive closure

The DOVfix branch of f0_D0Eapp:

```
dfix = dfun.arg2
denv_new0 = dfun.arg1
denv_new1 = \
    ENVcns(dfix.arg1, dfun, denv_new0)
denv_new2 = \
    ENVcns(dfix.arg2, darg, denv_new1)
return d0exp_evaluate(dfix.arg3, denv_new2)
```

Lookup order: parameter, function's own name, captured bindings.

Binding the function name to dfun lets the body call the same closure.

If the parameter and function names coincide, the parameter shadows it.

Factorial as an abstract syntax tree

```
factorial = D0Efix("fact", "n",  
    D0Eif0(  
        D0Eop2("<=", D0Evar("n"), D0Eint(0)),  
        D0Eint(1),  
        D0Eop2("*", D0Evar("n"),  
            D0Eapp(D0Evar("fact"),  
                D0Eop1("-1", D0Evar("n"))))))))  
  
d0exp_evaluate(D0Eapp(factorial, D0Eint(4)))  
# D0Vint(arg1=24)
```

Each call adds a new `n` binding and the same `fact` closure.

The calls descend through 4, 3, 2, 1, 0. Multiplication finishes as they return.

Conditionals evaluate only the selected branch

Body of f0_D0Eif0:

```
dexp1 = d0exp_evaluate(dexp.arg1, denv)
if not isinstance(dexp1, D0Vbtf):
    raise TypeError(f"D0Vbtf(...) expected: {dexp1}")
if dexp1.arg1:
    return d0exp_evaluate(dexp.arg2, denv)
else:
    return d0exp_evaluate(dexp.arg3, denv)
```

Despite its name, D0Eif0 expects a Boolean, not an integer zero test.

In factorial, the base case avoids evaluating the recursive branch.

Primitive operations check value kinds

Operators	Operand values	Result value
+1, -1	One D0Vint	D0Vint
+, -, *, /	Two D0Vints	D0Vint
<, >, <=, >=, ==, !=	Two D0Vints	D0Vbtf

- Binary operations evaluate both operands from left to right.
- The interpreter's / uses Python's floor division //. For example, $-7/3$ produces `D0Vint(-3)`.
- Wrong operand kinds and unsupported operators raise `TypeError`.

Pairs evaluate both components

The D0Epair branch:

```
dval1 = d0exp_evaluate(dexp.arg1, denv)
dval2 = d0exp_evaluate(dexp.arg2, denv)
return D0Vpair(dval1, dval2)
```

D0Epfst(e) and D0Epsnd(e) evaluate e, require a D0Vpair, then select one component.

Pairs can contain mixed kinds, nested pairs, and closures.

```
d0exp_evaluate(D0Epfst(
    D0Epair(D0Eint(7), D0Ebtft(True))))
# D0Vint(arg1=7)
```

Call-by-value makes unused computations matter

Let `bad` be an expression that divides by zero:

```
bad = D0Eop2("/", D0Eint(1), D0Eint(0))
```

```
D0Eapp(D0Elam("x", D0Eint(7)), bad)
```

```
D0Elet("x", bad, D0Eint(7))
```

```
D0Epfst(D0Epair(D0Eint(7), bad))
```

Evaluating any of these three expressions raises `ZeroDivisionError`.

By contrast, this conditional produces `D0Vint(7)`:

```
D0Eif0(D0Ebtft(True), D0Eint(7), bad)
```

Errors and implementation boundaries

- An unbound variable returns the sentinel `DOV000()`.
It can propagate as a result or become a pair component.
- Invalid application, projection, or operand kinds raise `TypeError`.
- Application evaluates the argument before checking the function kind.
An argument's error can therefore occur first.
- Dataclass annotations assume correctly formed ASTs and environments.
They do not validate constructor arguments.
- Evaluation uses Python recursion, including for tail calls.
Deep calls can exceed the recursion limit. Some terms never terminate.

Questions for tracing the interpreter

- 1 What happens if application extends the caller's environment instead of `dfun.arg1`?
Revisit the example with `x = 10` and `x = 100`.
- 2 Why does `D0Vfix` application add the function's own binding?
- 3 What does this expression return?

```
D0Eapp(  
    D0Eapp(D0Elam("x", D0Elam("y", D0Evar("x"))),  
           D0Eint(10)),  
    D0Eint(20))
```

Relevant examples and checks: `TEST/test03_lambda1.py`

Run with `python3 -B TEST/test03_lambda1.py`.